# Serial Communication, I2C and SCCB

## Pre-lab lecture for Lab 1: reading the OV7670 ID registers

Parsa Mandooie · FPGA Lab · Mehr 1405

School of Electrical and Computer Engineering, College of Engineering
University of Tehran

Design, Verification & Debugging
of Embedded Systems LAB

DVDES

# Outline

- Lab overview
- Serial communication and UART
- I2C
- SCCB
- Next: Lab 1

# The four experiments

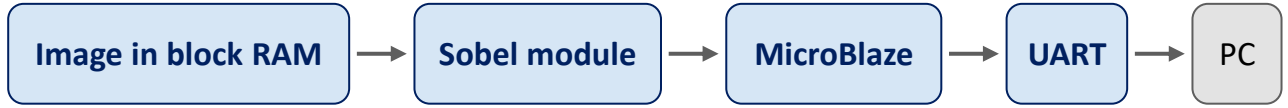
**Lab 1**
Camera control




**Lab 2**
Sobel in software, timed



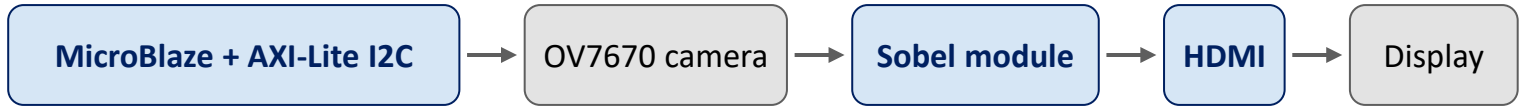

**Lab 3a**
Sobel in hardware, timed




**Lab 3b**
Live camera to display




**Lab 4**
Live edge detection




Inside the FPGA  Outside the FPGA

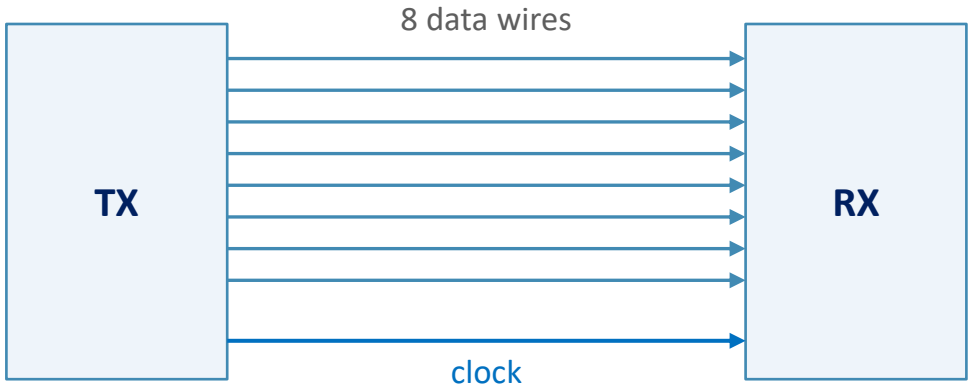
DVDES

# Outline

- Lab overview
- Serial communication and UART
- I2C
- SCCB
- Next: Lab 1

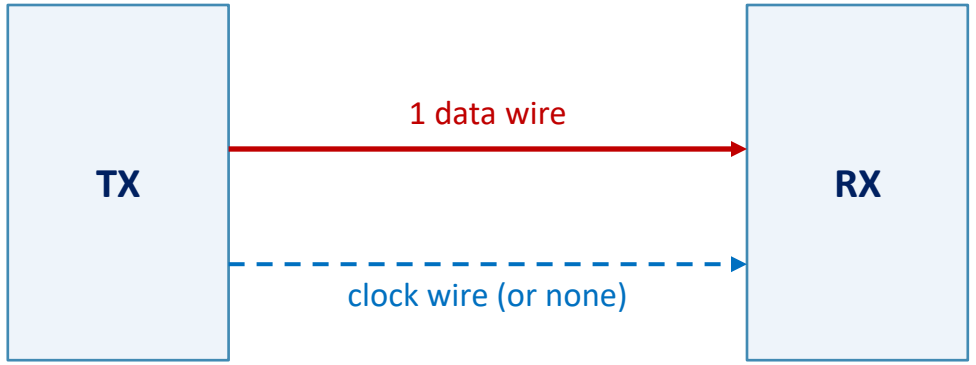
# Parallel vs serial

## Parallel




- One pin and one trace per bit, plus a clock
- Bits arrive at slightly different times (skew)
- Skew and crosstalk limit speed and distance

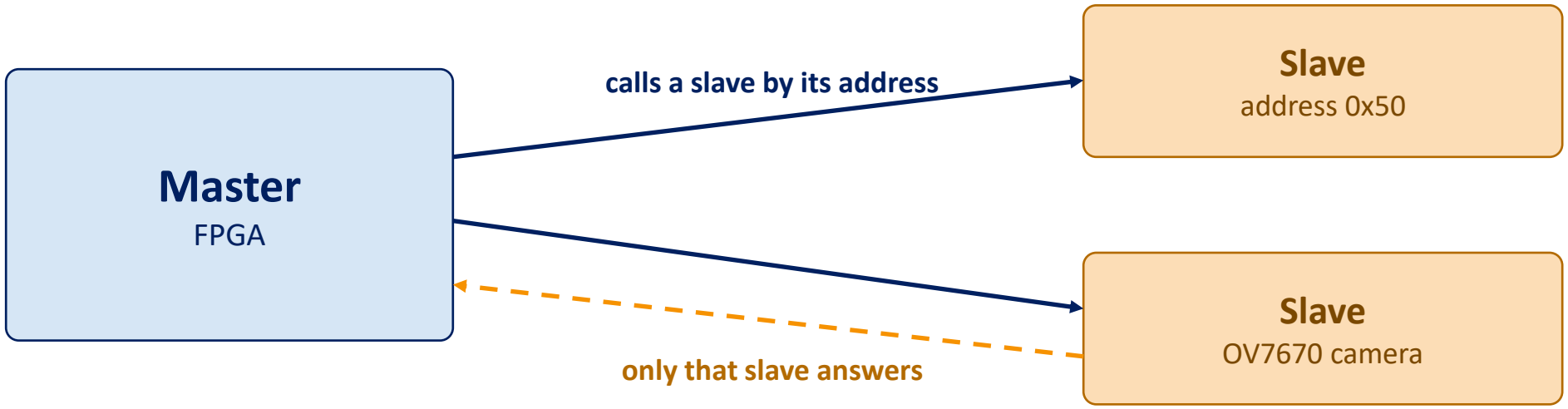
## Serial




- One bit at a time on one or two wires
- Few pins, cheap connectors and cables
- No skew between data bits: wins off-chip

DVDES

# Master and slave




**Master**
starts every transfer, picks the slave, drives the clock

**Slave**
waits until it is called, then receives or sends

**Role is not direction**
in a read the slave sends, still on the master's clock

# Ways to classify a serial link

## Clocking

**Asynchronous:** no clock wire; both ends agree on a bit rate (UART)

**Synchronous:** the master drives a clock wire, in both directions; a slave that sends data uses the master's clock too (I2C)

## Topology

**Point-to-point:** exactly two devices (UART)

**Bus:** many devices share the wires, so they need addresses (I2C)

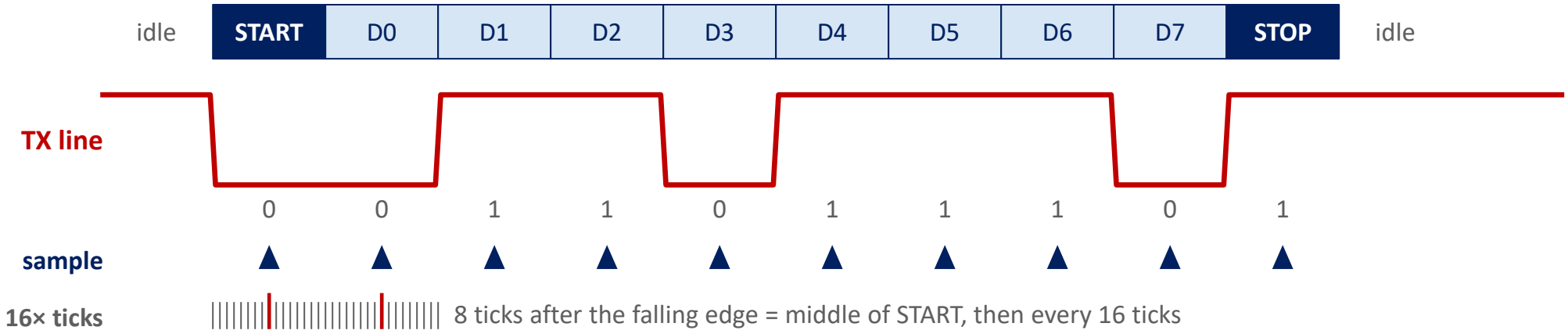
## Direction

**Simplex:** one way only

**Half duplex:** both ways, one at a time (I2C)

**Full duplex:** both ways at once (UART)

# UART frame




- Idle is high; the START bit (low) marks the beginning of a frame
- Data goes LSB first: 5 to 8 bits, parity, then 1 or 2 STOP bits (high)
- **Parity (optional):** makes the count of 1s even or odd; detects one flipped bit
- No clock wire: both ends are configured for the same baud rate
- **Example:** 0x76 as 8N1 at 115 200 baud; one bit lasts 8.68 µs

DVDES

# Where UART stops

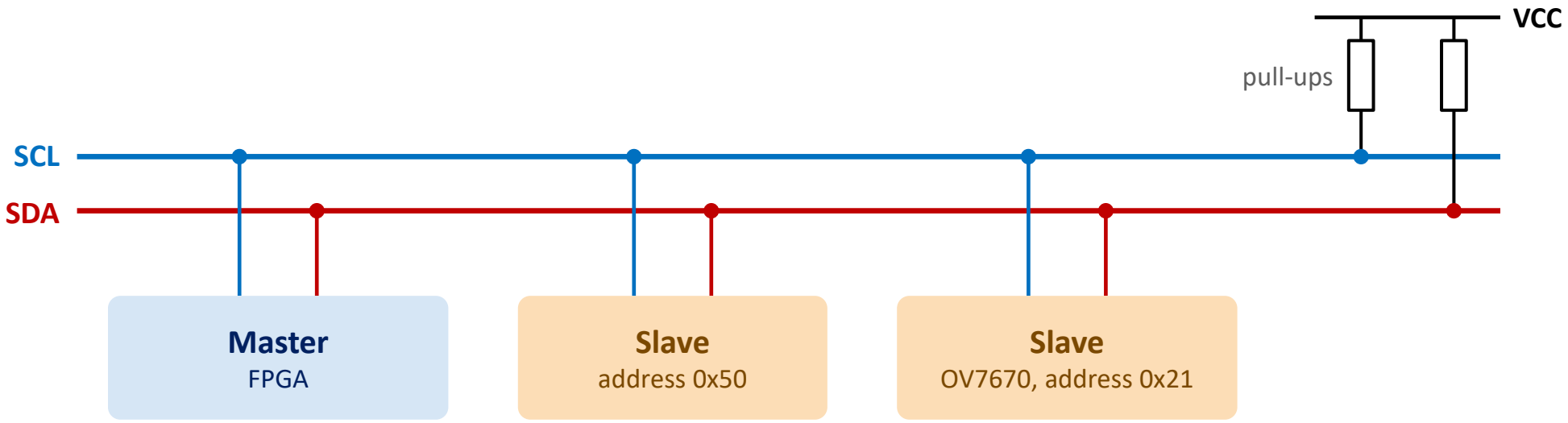
| UART | What a shared bus needs |
| --- | --- |
| Only two devices per link | → **Many devices on the same two wires** |
| No addressing | → **Every transfer starts with an address** |
| No shared clock: baud rates must match within about 5 % | → **The master sends the clock with the data** |
| The receiver cannot confirm a byte | → **An acknowledge bit after every byte** |

DVDES

# Outline

- Lab overview
- Serial communication and UART
- I2C
- SCCB
- Next: Lab 1

# I2C at a glance




**2 wires**
SCL = clock, SDA = data

**1 master**
starts every transfer, makes the clock

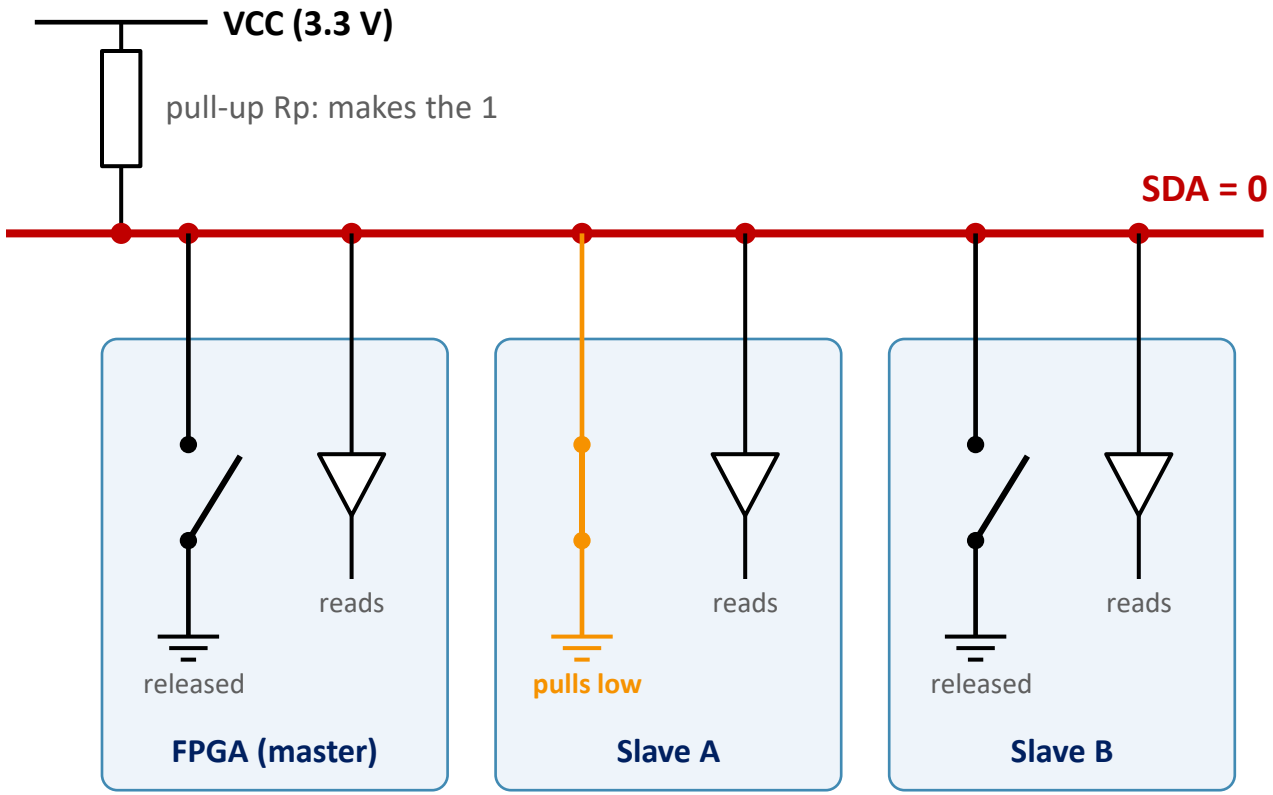
**7-bit address**
one per slave, sent first

**100 / 400 kbit/s**
Standard-mode / Fast-mode

*UM10204 Rev. 7 (2021) calls them controller and target; the lab material keeps master and slave.*

DVDES

# Open-drain and pull-ups




*SCL is wired exactly the same way.*

- A device can only pull the line low, or release it
- The resistor makes the 1; a transistor makes the 0
- Line = AND of all devices (wired-AND)

**Never drive a hard 1.** A 1 against another device's 0 is a short from VCC to GND.

**Lab boards:** no resistors on the camera connector; FPGA internal pull-ups are enabled.

DVDES

# Idle bus, and who drives what

| Moment | SCL | SDA |
|---|---|---|
| Bus idle | released (high) | released (high) |
| START, repeated START, STOP | master | master |
| Address byte, data written by the master | master | master |
| ACK after those bytes | master | slave |
| Data read from the slave | master | slave |
| ACK / NACK after read data | master | master |

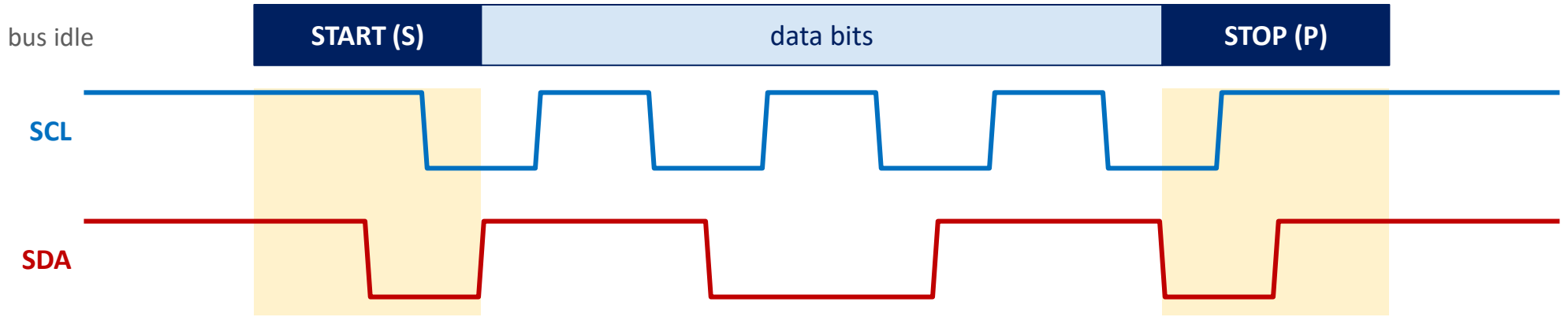
Master drives  Slave drives

**Idle = released**
Nobody pulls low, so the pull-ups hold both lines high.

**Release to listen**
Whenever the slave drives SDA, the master must have released it and only read the pin.

DVDES

# START and STOP



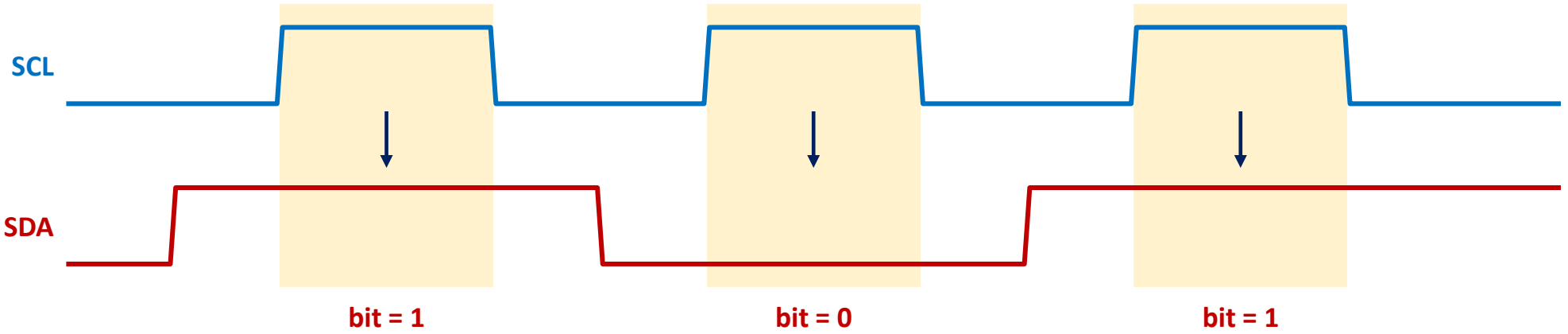

**START:** SDA falls while SCL is high

**STOP:** SDA rises while SCL is high

- Everywhere else, SDA may change only while SCL is low
- Only the master makes START and STOP; the bus is busy between them

DVDES

# Transferring one bit



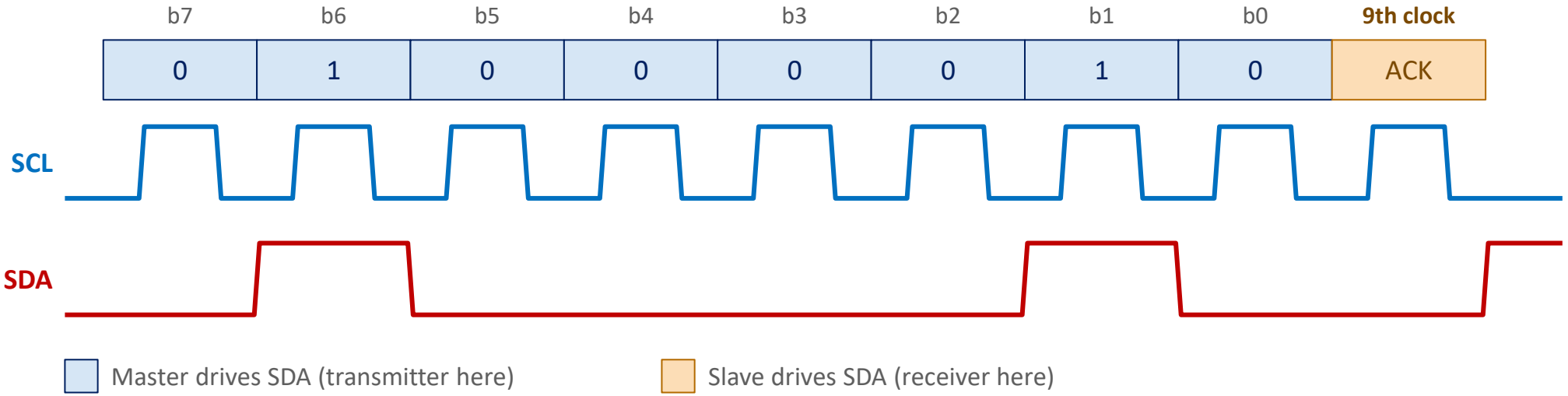

SCL high: SDA is stable, the receiver samples

SCL low: the transmitter may change SDA

- **Transmitter:** change SDA only while SCL is low
- **Receiver:** sample SDA while SCL is high
- A change of SDA while SCL is high is never data: it is START or STOP

DVDES

# One byte: 8 bits + ACK



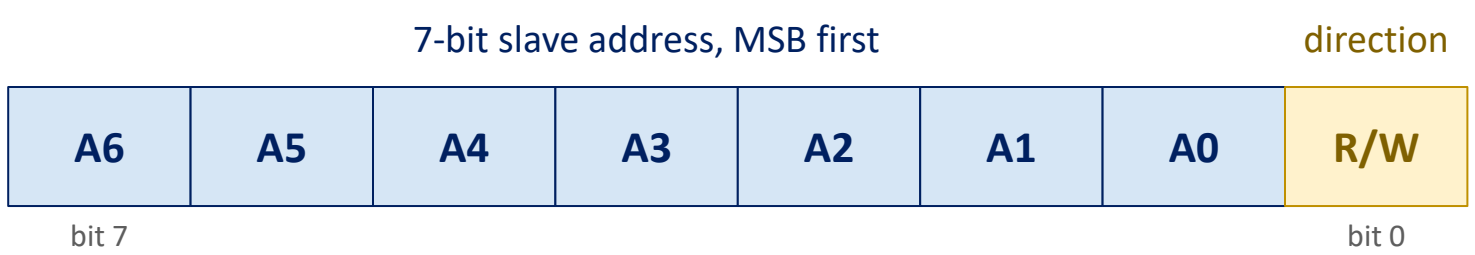

- 8 data bits, MSB first
- 9th clock: the transmitter releases SDA and the receiver answers

**ACK = 0**
receiver pulls SDA low: go on

**NACK = 1**
SDA stays high: nobody there, or stop

DVDES

# The address byte




**R/W = 0** write
**R/W = 1** read

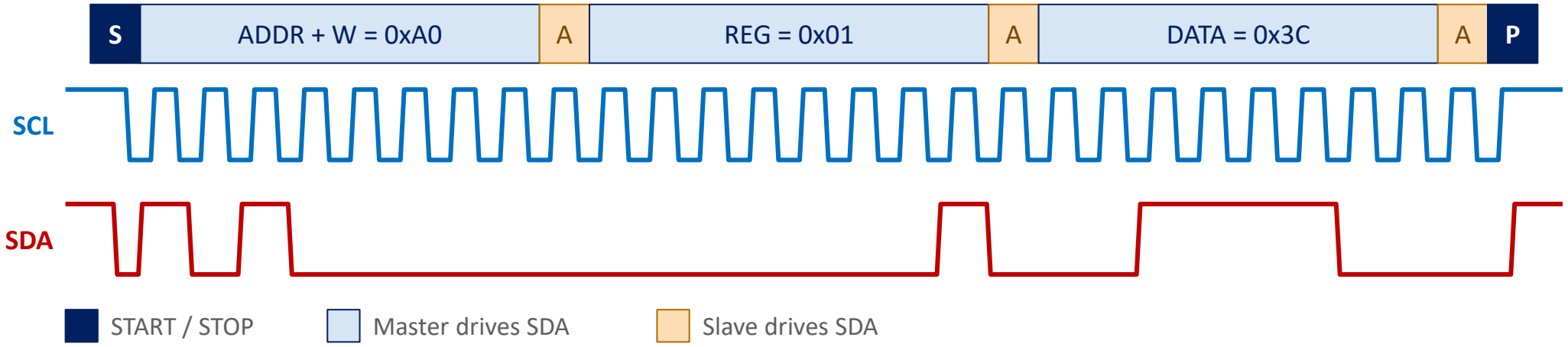
| Device | 7-bit address | Binary | Write byte (R/W = 0) | Read byte (R/W = 1) |
|---|---|---|---|---|
| **Lab I2C slave model** | 0x50 | 101 0000 | 0xA0 | 0xA1 |
| **OV7670 camera** | 0x21 | 010 0001 | 0x42 | 0x43 |

- Byte on the wire = (address << 1) | R/W
- Datasheets quote either the 7-bit address or the two 8-bit bytes: check which

DVDES

# Write transaction

Write 0x3C to register 0x01 of the slave at address 0x50

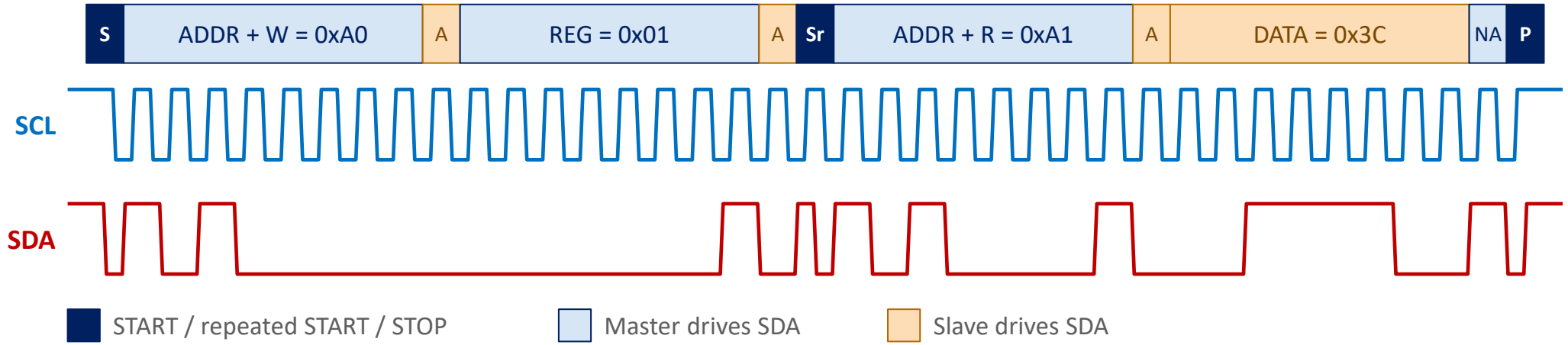



- The slave ACKs every byte the master sends
- The byte after the address selects the register (a pointer inside the slave)
- STOP ends the transfer and frees the bus

DVDES

# Read transaction

Read register 0x01 of the slave at address 0x50 (it returns 0x3C)

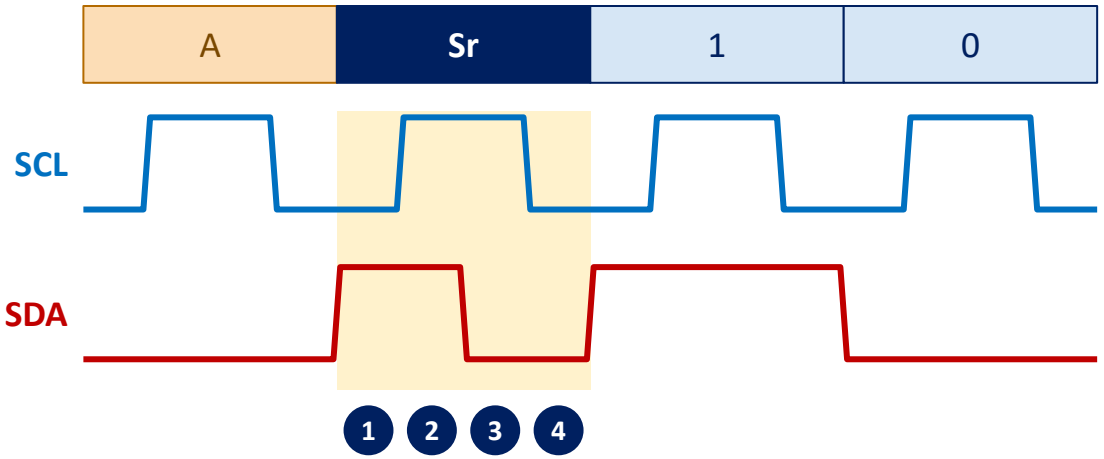



**Two halves.** The write half sets the register pointer; the read half fetches the data.

**NACK on the last byte.** It means “no more”: the slave releases SDA, so the master can make the STOP.

DVDES

# Repeated START




**1** SCL is low: release SDA, it goes high

**2** Release SCL, it goes high

**3** Pull SDA low while SCL is high: this is the START

**4** Pull SCL low and continue with the next byte

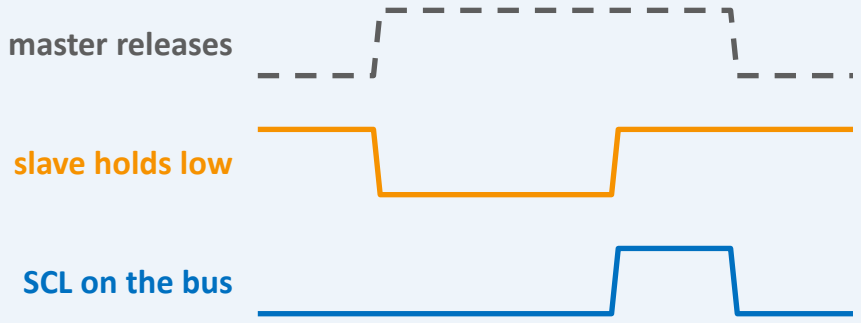
- A START with no STOP before it
- The bus stays busy: no other master can cut in
- Used to change direction, write to read, in one transfer
- To the slave, S and Sr look the same

**SCCB does not define a repeated START.** See Part 3.

DVDES

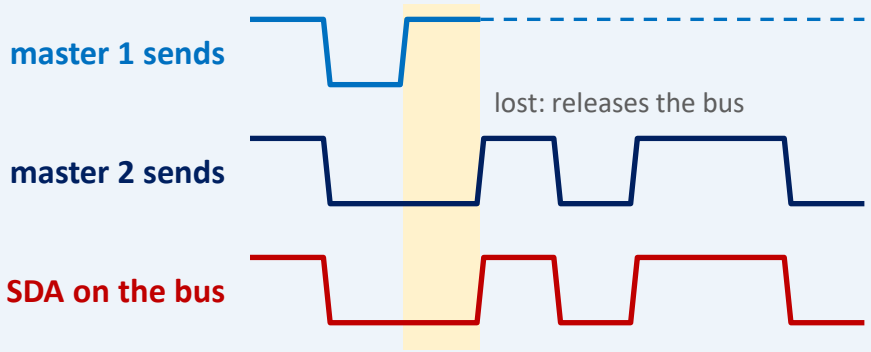
# Clock stretching and arbitration

## Clock stretching




A slave holds SCL low to make the master wait

## Arbitration

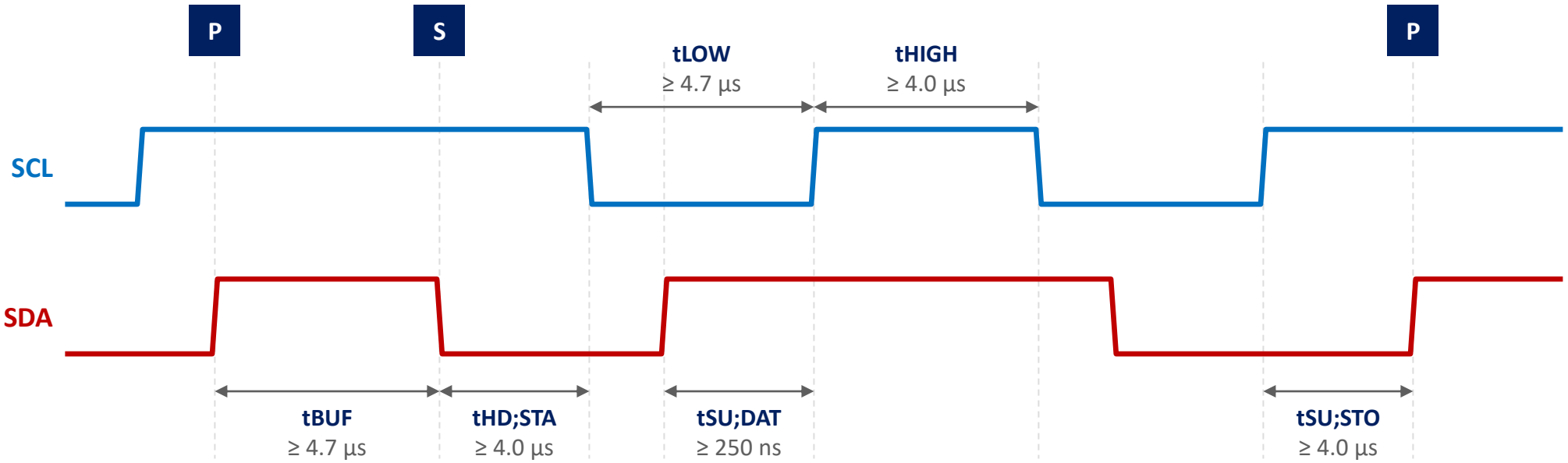



Two masters start together; the one that sends 1 but reads 0 backs off

**Not used in the lab:** 1 master, and the camera's clock pin is an input only.

DVDES

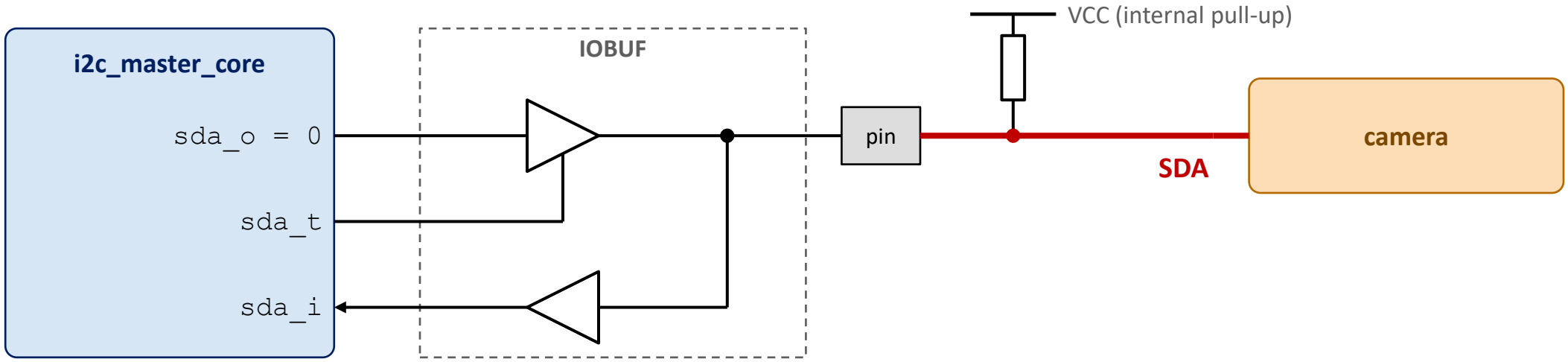
# Timing parameters by name




- Values are the Standard-mode (100 kHz) minimums from UM10204
- Four equal phases at 100 kHz: tLOW = tHIGH = 5 µs, tSU;DAT = 2.5 µs
- START hold and STOP setup become 2.5 µs: fine for the OV7670 (needs 0.6 µs)

DVDES

# The tri-state pin in the FPGA




| sda_t | Driver | SDA line |
|---|---|---|
| **1** | off (released) | 1, from the pull-up |
| **0** | on, drives sda_o = 0 | 0 |

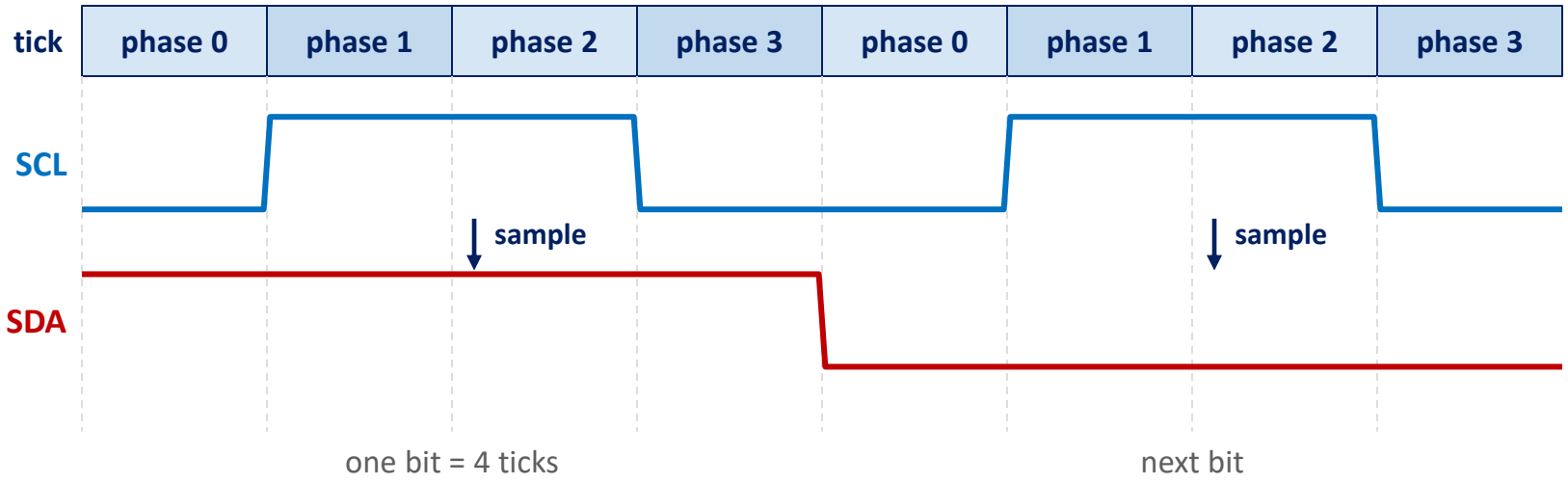
**In simulation (testbench only)**

```
tri1 scl, sda;     // wire with pull-up
assign sda = sda_t ? 1'bz : sda_o;
```

*Always read the bus from sda_i, never from what you meant to drive. SCL uses the same three signals.*

DVDES

# How the master generates one bit




**0**
SCL low: set SDA

**1**
SCL rises

**2**
SCL high: sample SDA

**3**
SCL falls

**f(SCL) = f(clk) / (4 × clk_div)**
100 MHz / (4 × 250) = 100 kHz

DVDES

# Transactions as byte operations

**Write 0x3C to register 0x01 (slave 0x50)**

| # | On the bus | tx_data | gen_start | stop_bit | rw_bit | nack_on_read |
|---|---|---|---|---|---|---|
| 1 | S, 0xA0, A | 0xA0 | 1 | 0 | 0 | – |
| 2 | 0x01, A | 0x01 | 0 | 0 | 0 | – |
| 3 | 0x3C, A, P | 0x3C | 0 | 1 | 0 | – |

**One start_txn pulse = one byte**
START and STOP are flags on that byte.

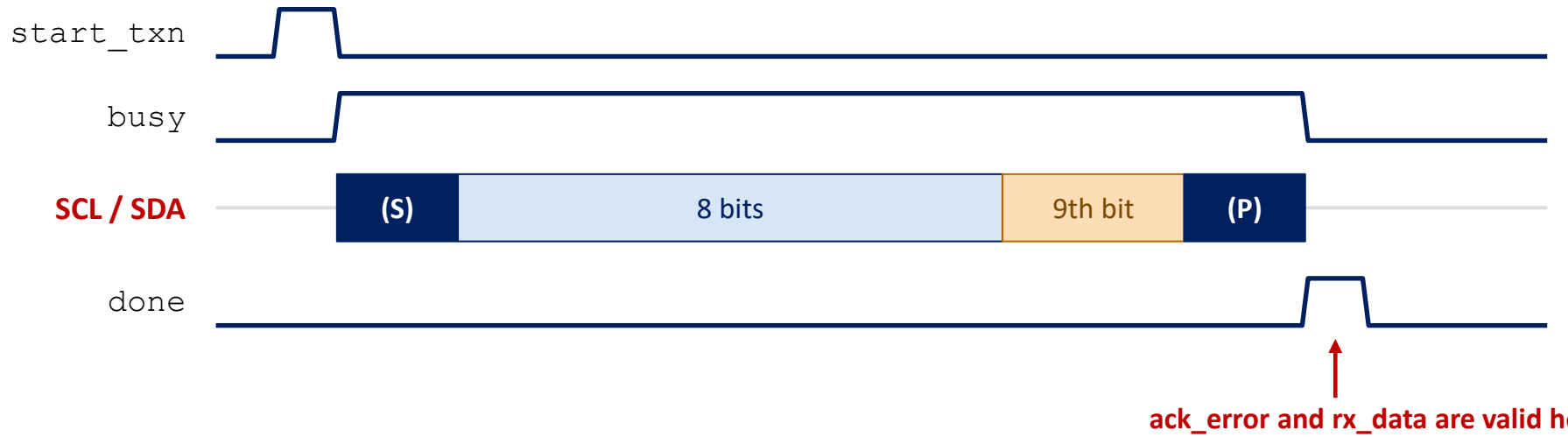
**Read register 0x01 (slave 0x50)**

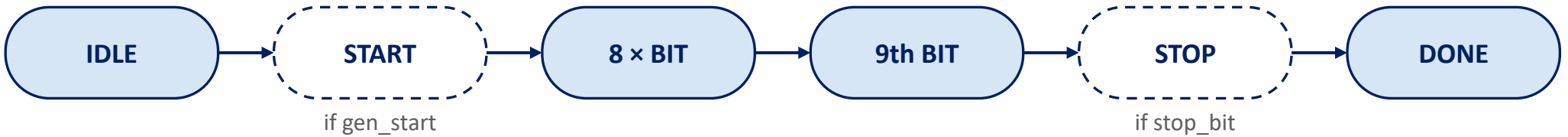
| # | On the bus | tx_data | gen_start | stop_bit | rw_bit | nack_on_read |
|---|---|---|---|---|---|---|
| 1 | S, 0xA0, A | 0xA0 | 1 | 0 | 0 | – |
| 2 | 0x01, A | 0x01 | 0 | 0 | 0 | – |
| 3 | Sr, 0xA1, A | 0xA1 | 1 | 0 | 0 | – |
| 4 | data, NA, P (rx_data = 0x3C) | – | 0 | 1 | 1 | 1 |

**Row 3:** the address byte 0xA1 is still a byte you send, so rw_bit = 0.

DVDES

# Inside one byte operation



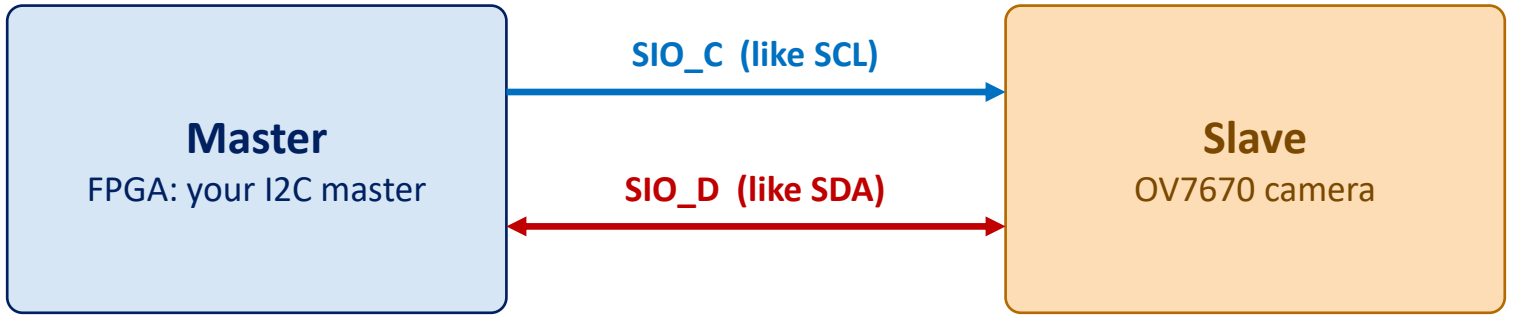

**A sketch, not the solution.** Hint: with no STOP, keep SCL low between bytes, or the slave sees a STOP.

DVDES

# Outline

- Lab overview
- Serial communication and UART
- I2C
- SCCB
- Next: Lab 1

# SCCB: the camera control bus



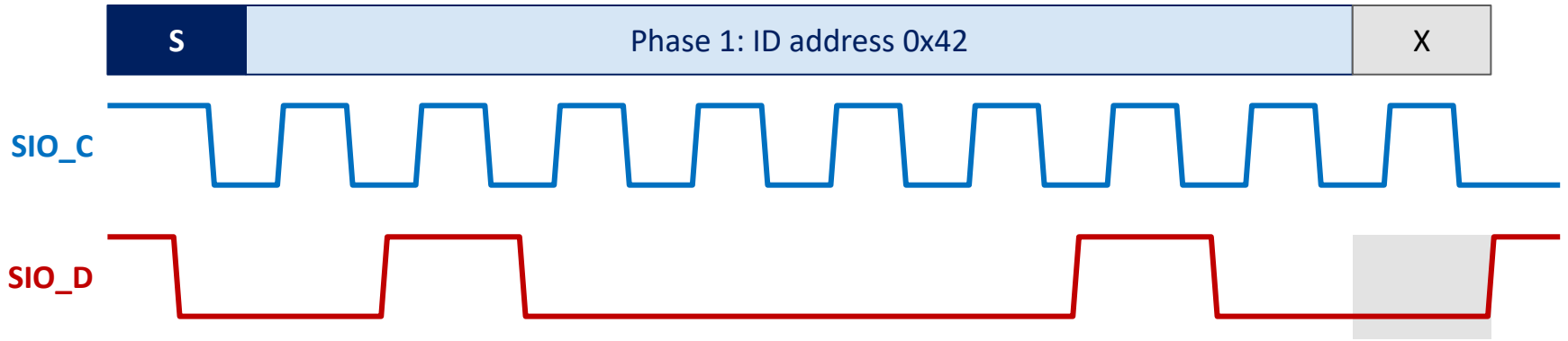

- OmniVision's bus for reading and writing sensor registers
- Same START and STOP, 8 bits MSB first, 7-bit ID address + R/W
- 2-wire version: one master, one slave
- Up to 400 kHz for the OV7670; the lab uses 100 kHz

**Why it looks like I2C**

It was designed to be driven by an ordinary I2C master. The wires and the bits are the same; the rules about the 9th bit and the transaction shapes are not.

DVDES

# Phases and the 9th bit




**A phase**

8 bits, MSB first, plus a 9th bit. A transmission has 2 or 3 phases.

**9th bit = X**

Don’t-care in every phase the master sends. The master need not check it.

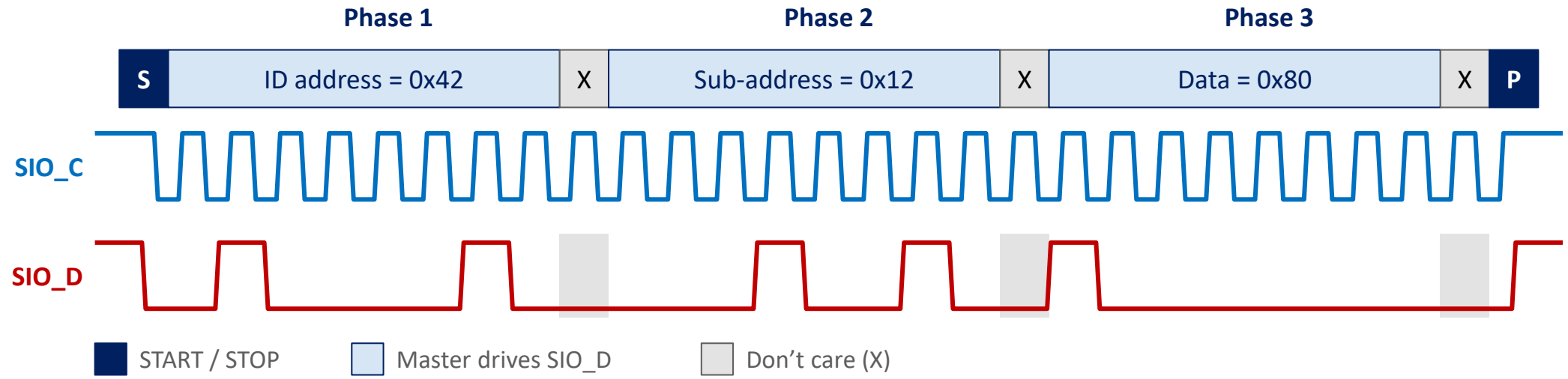
**9th bit = NA**

In the data phase of a read, the master drives 1 (like an I2C NACK).

*In practice the OV7670 pulls the X bit low, so an I2C master reads it as an ACK.*

DVDES

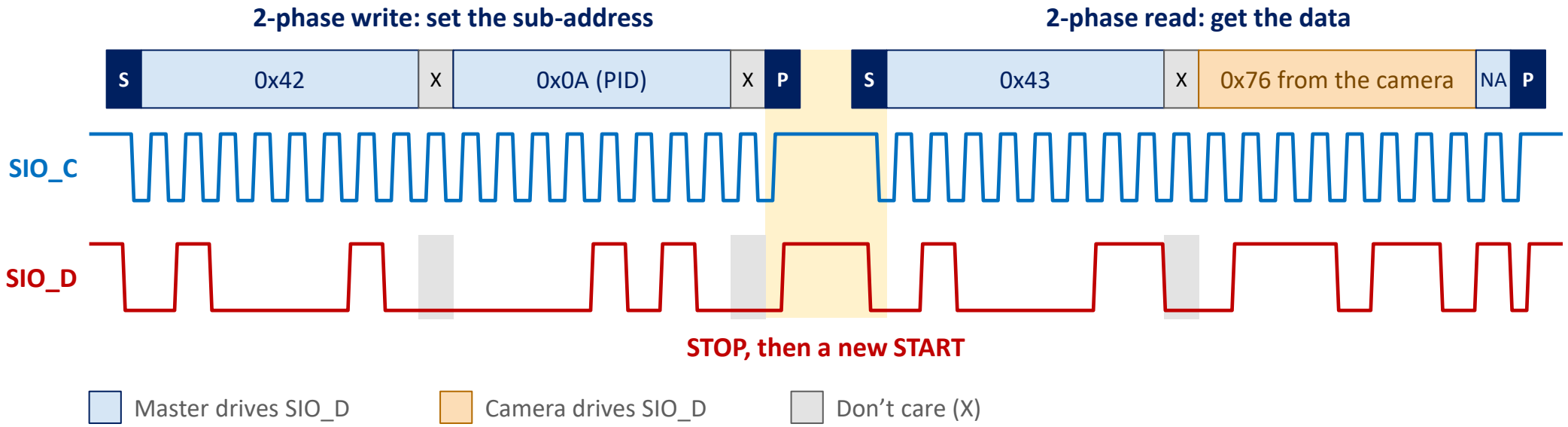
# 3-phase write




- Same shape as an I2C write, with X in place of ACK
- At most 3 phases per transmission: exactly one data byte per write
- **Example:** write 0x80 to COM7 (0x12), the software reset

DVDES

# Register read: two transmissions




**No repeated START.** Every transmission ends with a STOP before the next one begins.

**Example:** read PID at sub-address 0x0A; the camera returns 0x76.

DVDES

# I2C vs SCCB

| | I2C | SCCB (OV7670) |
|---|---|---|
| **9th bit** | ACK / NACK from the receiver | Don't-care (X); NA after read data |
| **Repeated START** | Yes | Not defined: STOP, then START |
| **Data bytes per write** | Any number | Exactly one (3 phases at most) |
| **Register read** | One transfer: write, Sr, read | 2-phase write, STOP, 2-phase read |
| **Devices** | Many, each with an address | One slave (2-wire version) |
| **Speed** | 100 / 400 kbit/s and faster modes | Up to 400 kHz |
| **Clock stretching** | Optional | None: SIO_C is an input of the camera |

# OV7670: what you need to know

## Before the first SCCB access

1. **XCLK running**
   24 MHz in the lab (10 to 48 MHz allowed)
2. **RESET# = 1**
   active-low reset released
3. **PWDN = 0**
   not in power-down

**ID address:** 0x42 to write, 0x43 to read

| Register | Sub-address | Value |
|---|---|---|
| **PID (product ID)** | **0x0A** | **0x76** |
| **VER (version)** | **0x0B** | **0x73** |
| **MIDH (manufacturer ID)** | 0x1C | 0x7F |
| **MIDL (manufacturer ID)** | 0x1D | 0xA2 |
| **COM7** | 0x12 | bit 7 = 1: reset all registers |

*After a reset through COM7, wait at least 1 ms before the next access.*

DVDES

# ID check: eight byte operations

| # | On the bus | tx_data | gen_start | stop_bit | rw_bit | nack_on_read | Result |
|---|---|---|---|---|---|---|---|
| 1 | S, 0x42, X | 0x42 | 1 | 0 | 0 | – | |
| 2 | 0x0A, X, P | 0x0A | 0 | 1 | 0 | – | sub-address = PID |
| 3 | S, 0x43, X | 0x43 | 1 | 0 | 0 | – | |
| 4 | data, NA, P | – | 0 | 1 | 1 | 1 | **rx_data = 0x76** |
| 5 | S, 0x42, X | 0x42 | 1 | 0 | 0 | – | |
| 6 | 0x0B, X, P | 0x0B | 0 | 1 | 0 | – | sub-address = VER |
| 7 | S, 0x43, X | 0x43 | 1 | 0 | 0 | – | |
| 8 | data, NA, P | – | 0 | 1 | 1 | 1 | **rx_data = 0x73** |

**PID = 0x76 and VER = 0x73** → the 1-bit output goes high and the LED turns on

DVDES

# Outline

- Lab overview
- Serial communication and UART
- I2C
- SCCB
- Next: Lab 1

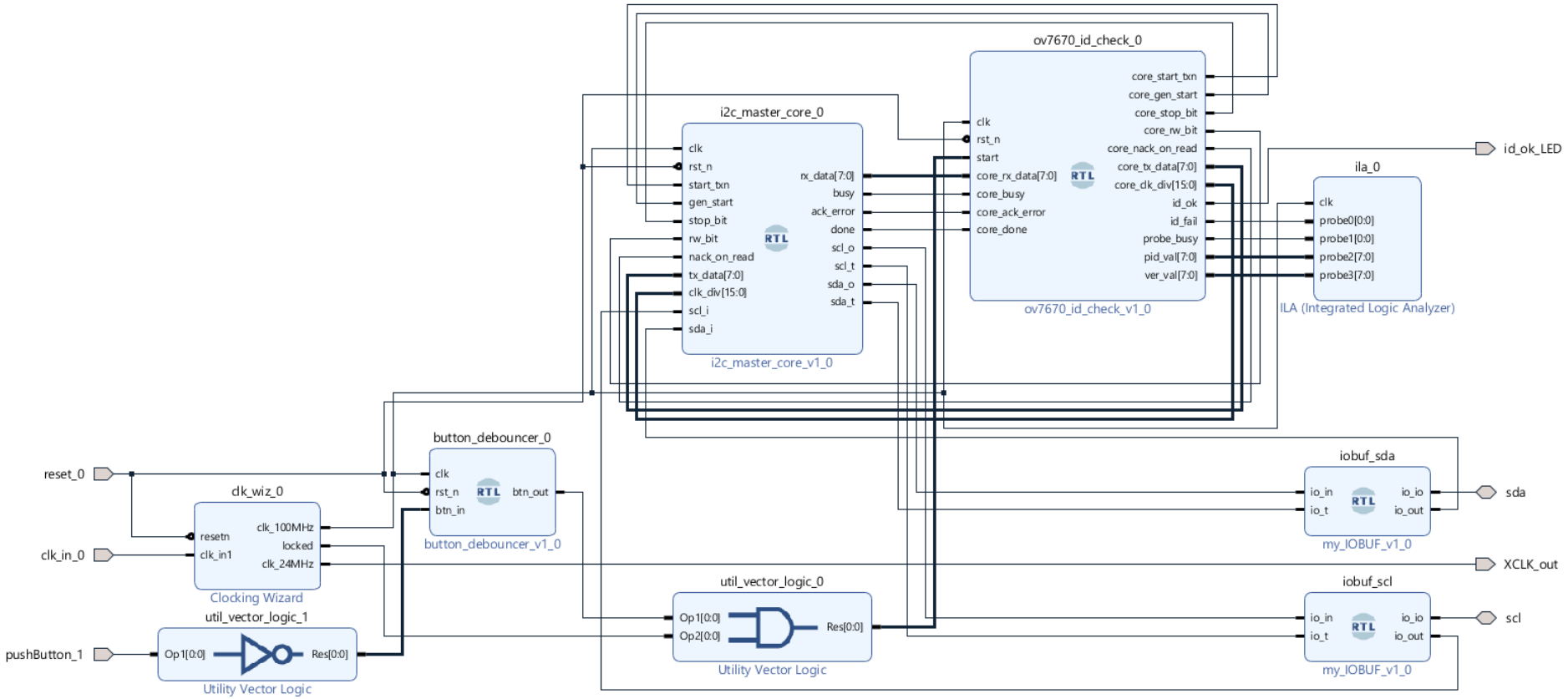
# The path through Lab 1




- The step-by-step instructions are in the Lab 1 handout
- Both models are simulation-only and print a message when the protocol is broken

DVDES

# The Lab 1 design in Vivado




**You write:** i2c_master_core and ov7670_id_check. **The handout sets up:** Clocking Wizard, debouncer, IOBUFs, ILA.

DVDES

# Protocol mistakes to avoid

**Driving a hard 1**
Release the line; never output a 1

**No pull-up**
Z or X in simulation, a floating line on the board

**Changing SDA while SCL is high**
The slave sees a START or a STOP

**Wrong ACK polarity**
ACK = 0 (pulled low), NACK = 1 (released)

**Forgetting the R/W bit**
Send 0x42 / 0x43, not the 7-bit 0x21

**Repeated START with the camera**
SCCB: STOP, then a new START

**No XCLK**
The camera does not answer without its clock

**The simulation models report most of these as error messages.**

DVDES

# References


- [1] NXP Semiconductors, “UM10204: I2C-bus specification and user manual,” Rev. 7.0, Oct. 2021.
- [2] OmniVision Technologies, “Serial Camera Control Bus (SCCB) Functional Specification,” version 2.2, June 2007.
- [3] OmniVision Technologies, “OV7670/OV7171 CMOS VGA CameraChip,” datasheet, version 1.4, Aug. 2006.
- [4] FPGA Lab, “Lab 1” instruction handout.